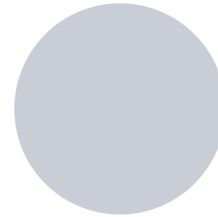


Connect an LLM.

Day 2 — the model decides. Your code executes.



First: what do you remember?

No slides needed for the answers — you built them.

1

What are the TWO things your server can do?

`read_file` · `get_weather` — the tools you wrote.

2

You change `server.py` and nothing changes. Why?

The server must be restarted.

3

Who should decide which tool to call — the programmer or the LLM?

Debate it. 30 seconds. Then next slide.

Who decides which tool to call?

The LLM. Always.

That is the entire architecture of Day 2.

But — the LLM cannot EXECUTE anything. It only names the tool and the arguments. Your code does the calling. Hold that thought.

The 4-step loop. (Watch step 4.)

The step almost everyone misses: the tool result goes BACK to the LLM.



Discover

“What can you do?” → server returns the tool list.



Decide

LLM reads the docstrings, names a tool + arguments.



Execute

YOUR code calls `session.call_tool()`. The server runs it.



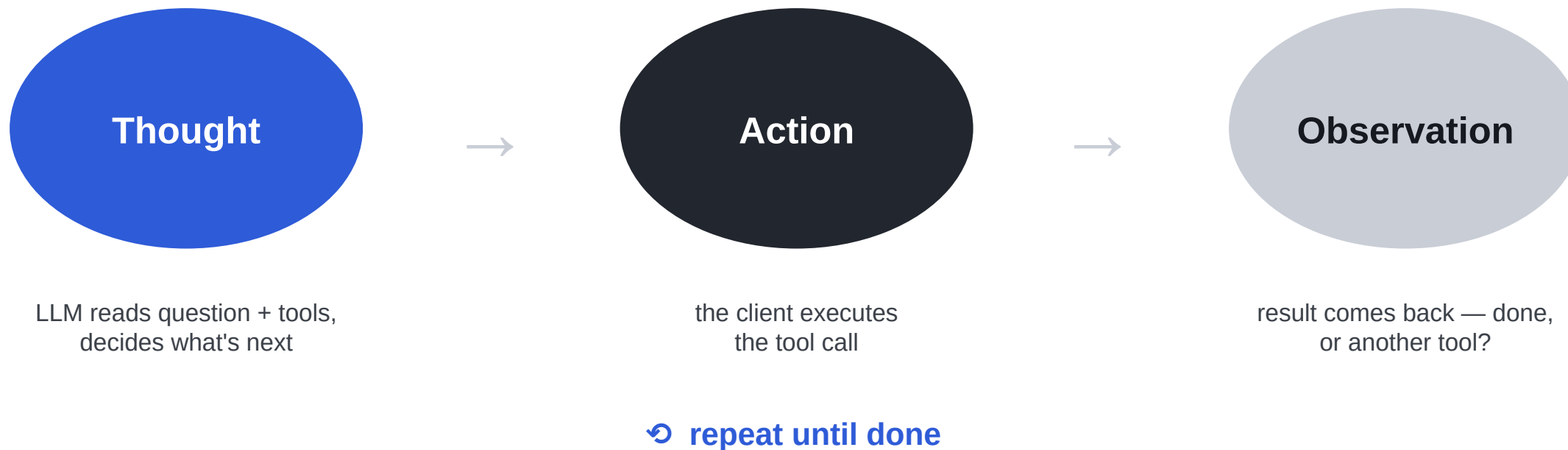
Synthesize

Result goes back to the LLM → it writes the final answer.

This loop is every AI agent. LangChain, LangGraph, smolagents — all this loop, packaged differently. In a moment, we write it ourselves.

The loop has a name: TAO.

Thought → Action → Observation — repeat until done.



client.py runs ONE cycle. An agent runs many — until it has enough to answer. That's tomorrow.

Two kinds of reasoning.

You're about to build the second one.

Chain-of-Thought (CoT)

Reasons step-by-step — in text only.

“Step 1: $12 \times 4 = 48$. Step 2: ...”

Use for: logic, math, analysis.

ReAct (Reason + Act)

Reasons AND calls tools.

“I need weather → call get_weather → now answer.”

Use for: anything needing live data or actions.

An LLM that reasons an LLM that ACTS.

Reasons or acts? Same model — here's the tell.

It's not two models. Check ONE thing to know which the model just did: `tool_calls`.

```
tool_calls = response.choices[0].message.tool_calls

if not tool_calls:
    ... # REASONED — answered from its own memory, no tool
else:
    ... # ACTED — named a tool+ its arguments
```

REASONED `tool_calls` is `None`
“What is the capital of France?”

ACTED `tool_calls` is populated
“Weather in Tel Aviv right now?”

Same model, every question. It reasons OR acts case by case — and your docstrings tip the decision.

But can this model act at ALL?

Tool-calling isn't universal. Two checks before you trust a model with tools.

1 · Ask Ollama

```
ollama show mistral:7b
```

Look for “tools” in the Capabilities list.

2 · Just test it

```
give it a tool+ a question  
that needs one
```

tool_calls comes back?
It can act.

Size is the tell you already hit. qwen3:0.6b REASONS fine but ACTS unreliably — tool-calling is the first ability to shrink with model size. Safe at ~3B+: **llama3.2 · qwen2.5/3 · mistral.**

Rule of thumb: a “reasoning model” and an “acting model” are the same thing — a tool-capable model, with tools wired in.

Enough theory.

Let's write the client.

The four steps — now in Python. Type them with me.

DO NOW — predict first (60 seconds)

You ask: “what's in my sales.csv?” The LLM has a `read_file` tool.

On your card: write the STEPS that must happen between your question and the answer.

1. _____
2. _____
3. _____
4. _____

You already saw the 4 steps — now map them to the code we're about to write.

The entire integration: three lines.

OpenAI Python SDK, pointed at Ollama. The `api_key` is required by the library — Ollama ignores it.

```
from mcp import ClientSession
from mcp.client.streamable_http import streamable_http_client
from openai import OpenAI

MCP_SERVER_URL = "http://127.0.0.1:8000/mcp/"
MODEL = "llama3.2:3b"

lm = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
```

Why llama3.2:3b and not qwen3:0.6b? Tool-calling is one of the first abilities that shrinks with model size. 0.6b answers questions fine — but picks tools unreliably.

Wait — why “OpenAI” for a local model?

It's not OpenAI the company. It's OpenAI the API SHAPE — the one everyone copied.

Ollama speaks it

Ollama exposes `/v1/chat/completions`. The standard SDK just works — point `base_url` at localhost, pass a dummy key. No account, no cost.

It's portable

`tools=[...]`, `tool_choice`, and `response.tool_calls` are the same contract at Groq, Together, vLLM, Mistral, DeepSeek. Learn it once.

One schema to teach

Every framework (LangChain, smolagents...) hides this exact format inside. You're learning the lingua franca, not a vendor.

```
# local today ...
OpenAI(base_url="http://localhost:11434/v1", api_key="ollama") # Ollama
# ... cloud tomorrow — change THREE things, same loop:
OpenAI(base_url="https://api.groq.com /openai/v1", api_key=KEY) # Groq
```

Not using OpenAI's models — using OpenAI's API shape, because everyone else adopted it too.

Step 1 — Discover.

Ask the server what it can do. No hard-coded endpoints, ever.

```
async with streamablehttp_client(MCP_SERVER_URL) as (r, w, _):  
    async with ClientSession(r, w) as session:  
        await session.initialize()  
        tools_result = await session.list_tools()  
        print([t.name for t in tools_result.tools])
```

```
$ python client.py  
['read_file', 'get_weather']
```

Those are YOUR tools from Day 1 — discovered, not imported.

Two dialects. One bridge.

MCP and OpenAI describe tools with different JSON. The conversion is mostly renaming.

```
def to_openai_tool(mcp_tool) -> dict:
    return {
        "type": "function",
        "function": {
            "name": mcp_tool.name,
            "description": mcp_tool.description or "",
            "parameters": {
                "type": "object",
                "properties": mcp_tool.input_schema.get("properties", {}),
                "required": mcp_tool.input_schema.get("required", []),
            },
        },
    }
```

inputSchema →
parameters.

That's 80% of the bridge.
Every framework you'll
ever use has this function
hidden inside it.

Step 2+3 — the LLM decides. It does NOT execute.

Watch what actually comes back: a decision in JSON. Not weather.

```
response = lm.chat.completions.create(
    model=MODEL,
    messages=messages,      # user:weather in TelAviv?
    tools=openai_tools,
    tool_choice="auto",
)
tool_calls = response.choices[0].message.tool_calls
```

```
The LLM returns:
name: "get_weather"
args: {"city":
      "TelAviv"}
```

It read both docstrings, chose one, provided the argument — and fetched NOTHING. **Executing is our job.**

Step 4 — Stop & Parse.

Stop the LLM · execute the real tool · resume with truth.

```
for tc in tool_calls:
    args = json.loads(tc.function.arguments)
    result = await session.call_tool(tc.function.name, arguments=args)
    # result is fed back into messages →
    # the LLM synthesizes the final answer from REAL data
```

Why not let the LLM write the result itself? Because it will — happily, confidently, wrongly. An LLM asked to continue past a tool call invents a plausible observation. Stop & Parse is the guardrail: the loop stops, real code runs, the true result re-enters the conversation.

Bonus: the for-loop already handles multiple tool calls in one response.

Live: watch the LLM's judgment.

Same code, three questions. The decision changes each time — because the docstrings drive it.

“What is the capital of France?”

no tool call — it already knows

“What is in the file notes.txt?”

calls `read_file`

“Weather in Tel Aviv right now?”

calls `get_weather`

The quality of the LLM's decision is exactly as good as the descriptions you write. **Docstrings are the steering wheel.**

DO NOW — break it, trick it, chain it (20 min)

Run `client.py` with your own questions. Answer three things:

1

Does the LLM always call a tool?

Find a question where it answers directly.

2

Can you trick it into calling the WRONG tool?

If yes — which words did the trick? (That's docstring engineering in reverse.)

3

What happens when a question needs BOTH tools?

Try: “what's in `notes.txt`, and what's the weather in Tel Aviv?”

Connection refused → start the Day 1 server first • model not found → `ollama pull llama3.2:3b` • new tool missing → restart server

Bonus: same loop, real database.

Neon runs an MCP server in the cloud. We change the URL — nothing else.

	client.py	neon_client.py
Transport	streamablehttp_client · local URL	HTTP · mcp.neon.tech (remote)
Auth	none	Bearer token header
Server	your Python file	Neon's cloud MCP server
The 4-step loop	identical	identical

```
python neon_client.py "How many rows are in the users table?"  
# the LLM chooses run_sql– docstrings written by the Neon team
```

The four rules of tool design.

Tight schemas

only the parameters the tool truly needs

Clear docstrings

say WHEN to use it, not just what it does

Return strings

text the LLM can read

One thing per tool

the LLM can chain calls itself

Today: you wrote the agent loop by hand — 50 lines, and you understand every one.

Tomorrow: a framework runs that loop for you, many cycles deep — and you build a server for a domain YOU care about.